

## **Trabalho Prático**

**Galeria de Imagens com Mapa e Persistência Local em SQLite**

# Objetivo Geral

Desenvolver um aplicativo React Native (Expo) que:

- Permita selecionar ou capturar imagens
- Associe cada imagem a uma localização geográfica
- Exiba as imagens em formato de galeria
- Mostre os pontos das imagens no mapa
- Persista os dados localmente em **SQLite**

## Contexto do Trabalho

Este trabalho integra conceitos das aulas de:

- Componentes React Native
- Consumo de APIs nativas (câmera e localização)
- Geolocalização e mapas
- Persistência local com SQLite

# Tecnologias obrigatórias

- React Native com Expo
- `expo-sqlite`
- `expo-image-picker` (ou `expo-camera` )
- `expo-location`
- `react-native-maps`

Instalação mínima:

```
npx expo install expo-sqlite expo-image-picker expo-location react-native-maps
```

## Requisitos Funcionais (Obrigatórios)

1. Permitir adicionar uma nova imagem (câmera ou galeria)
2. Solicitar e obter localização atual no momento do cadastro
3. Salvar no SQLite: título, URI da imagem, latitude, longitude, data
4. Listar todas as imagens em uma tela de galeria
5. Exibir uma tela de mapa com **marcadores** de todas as imagens
6. Ao tocar no marcador, mostrar título e miniatura
7. Permitir excluir um item da galeria



## Modelo mínimo do banco

Tabela obrigatória: photos

```
CREATE TABLE IF NOT EXISTS photos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  image_uri TEXT NOT NULL,  
  latitude REAL,  
  longitude REAL,  
  created_at TEXT NOT NULL  
);
```



## Fluxo esperado da aplicação

1. Usuário abre app
2. Toca em "Adicionar imagem"
3. Informa título
4. Escolhe/tira foto
5. App captura localização atual
6. App salva no SQLite
7. Galeria atualiza automaticamente
8. Mapa exibe marcador da nova imagem

# Telas mínimas esperadas

- **Tela 1: Galeria**
  - Lista/grid com imagem + título + data
  - Botão para adicionar nova imagem
- **Tela 2: Mapa**
  - Mapa com marcadores das imagens salvas
  - Callout com informações básicas

Navegação pode ser por abas, stack ou drawer.





## Requisitos Técnicos (Obrigatórios)

- Criar camada de acesso ao banco separada da UI
- Usar SQL parametrizado
- Tratar permissão negada (câmera/localização)
- Tratar erro de banco sem quebrar o app
- Garantir que os dados persistem ao fechar e abrir o app

## ★ Extras (Diferenciais)

- Edição de título da imagem
- Filtro por proximidade ou data
- Busca por título
- Cluster de marcadores no mapa
- Exportação/importação de backup local
- Interface mais elaborada e responsiva

# Entrega

Enviar:

1. Link do repositório (GitHub)
2. Vídeo curto (2 a 5 min) mostrando:
  - Cadastro de imagem com localização
  - Persistência no SQLite
  - Galeria funcionando
  - Mapa com marcadores
3. README com instruções de execução

## Critérios de Avaliação

| Critério                             | Peso |
|--------------------------------------|------|
| Funcionalidades obrigatórias         | 1,6  |
| Persistência correta com SQLite      | 0,8  |
| Integração com mapa e geolocalização | 0,8  |
| Organização de código e arquitetura  | 0,4  |
| Qualidade da apresentação/README     | 0,4  |

**Total:** 4,0

## **Itens que zeram requisito funcional**

- Não usar SQLite
- Não salvar latitude/longitude
- Não exibir mapa com marcadores
- App não persistir dados após reiniciar



## Sugestão de Cronograma

- Aula 1: modelagem + tabela + cadastro
- Aula 2: galeria + exclusão
- Aula 3: mapa + marcadores
- Aula 4: ajustes finais + apresentação

## Dicas finais

- Comece pela camada de banco e teste com dados simples
- Depois integre câmera/galeria e localização
- Por último, conecte no mapa
- Commits pequenos e frequentes ajudam muito

## **Resultado Esperado**

Um app mobile funcional que opere offline, com persistência local robusta e visualização georreferenciada das imagens.

Esse projeto simula uma demanda real de mercado para apps de vistoria, campo, turismo e logística.